

12-Dynamic Proxy(动态代理)

鸣谢：黑马程序员



💡 Tip

Previous Chapter(上一章): 11-Annotation(注解)

- [🔗 Markdown](#)
- [🔗 PDF](#)
- [🔗 HTML](#)

• 12-Dynamic Proxy(动态代理)

一、概述

二、代码演示

1.需求

2.创建代理对象

3.代码实现

3.1 定义 `Star` 接口和 `SuperStar` 实现类

3.2 定义代理工具类 `ProxyUtil`

3.3 编写测试类 `Test`

3.4 控制台输出

一、概述

- 如果某个对象的职责过多，可以通过代理来转移部分职责。
 - 代理必须具备对象委托其代理的方法。比如对象有 `run()` 方法想委托给代理处理，那么代理也必须具备 `run()` 方法。
 - Java中规定对象和代理都必须实现特定接口，以此保证这条原则的落实。
-

二、代码演示

1.需求

一位明星唱歌和跳舞时需要处理的业务有三个阶段：

- 准备阶段。
- 唱歌/跳舞阶段（核心业务）。
- 收尾阶段（可以回应观众，也可以什么都不做）。

但是对于这位明星来说，他最关心的是核心业务，而如果每次想要唱歌/跳舞时都要亲自处理这三个阶段的业务，就会导致职责过多。

这时候，我们可以考虑将明星的准备阶段和收尾阶段交由经纪人实现，也就是代理。

2.创建代理对象

`java.lang.reflet.Proxy` 类提供了为对象生成代理对象的方法：

```
1 public static Object newProxyInstance(  
2     ClassLoader loader,      // 指定使用哪个类加载器去加载生成的代理类  
3     Class<?>[] interfaces, // 指定接口，这些接口用于指定生成的代理具有什么方法  
4     InvocationHandler h      // 指定生成的代理对象要处理的业务  
5 )
```

3.代码实现

3.1 定义 `Star` 接口和 `SuperStar` 实现类

- `Star` 接口:

```
1 package proxy;
2
3 public interface Star {
4     String sing(String song);
5
6     void dance();
7 }
```

- `SuperStar` 实现类:

```
1 package proxy;
2
3 // 明星类
4 public class SuperStar implements Star {
5     private String name;
6
7     public SuperStar(String name) {
8         this.name = name;
9     }
10
11     @Override
12     public String sing(String song) {
13         System.out.println(name + " is singing: " + song);
14         return "thank you!";
15     }
16
17     @Override
18     public void dance() {
19         System.out.println(name + " is dancing");
20     }
21 }
```

3.2 定义代理工具类 ProxyUtil

⚠ Warning

此处使用的 `Proxy` 类是 `java.lang.reflect` 这个包下的，注意不要错选成其他包下的。

```

1  package proxy;
2
3  import java.lang.reflect.InvocationHandler;
4  import java.lang.reflect.Method;
5  import java.lang.reflect.Proxy;
6
7  public class ProxyUtil {
8      public static Star createProxy(SuperStar superStar) {
9          Star starProxy = (Star) Proxy.newProxyInstance(
10             ProxyUtil.class.getClassLoader(), // 指定使用哪个类加载器去
加载生成的代理类
11             new Class[]{Star.class},           // 指定接口，这些接口用于
指定生成的代理具有什么方法
12             new InvocationHandler() {          // 指定生成的代理对象要处
理的业务
13                 @Override
14                 public Object invoke(Object proxy, Method method,
Object[] args) throws Throwable {
15                     if (method.getName().equals("sing")) {
16                         System.out.println("prepare to sing");
17                     } else if (method.getName().equals("dance")) {
18                         System.out.println("prepare to dance");
19                     }
20                     return method.invoke(superStar, args); // 核心业
务交由对象处理
21                 }
22             }
23         );
24         return starProxy;
25     }

```

3.3 编写测试类 `Test`

```
1 package proxy;
2
3 public class Test {
4     public static void main(String[] args) {
5         SuperStar superStar = new SuperStar("Beyond");
6         Star starProxy = ProxyUtil.createProxy(superStar);
7
8         String reply = starProxy.sing("灰色轨迹");
9         System.out.println(reply);
10
11        starProxy.dance();
12    }
13 }
```

3.4 控制台输出

```
D:\CS-Softwares\JDK\JDK21\bin\java.exe "-javaagen
prepare to sing
Beyond is singing: 灰色轨迹
thank you!
prepare to dance
Beyond is dancing

Process finished with exit code 0
```